

AIAC Assignment 8.5

Name : k Nikshitha

Ht no: 2303A51692

Batch 24

Task Description #1 (Username Validator – Apply AI in Authentication Context)

• Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)` and then implement the function using Test-Driven Development principles.

• Requirements:

- o Username length must be between 5 and 15 characters.
- o Must contain only alphabets and digits.
- o Must not start with a digit.
- o No spaces allowed.

Example Assert Test Cases:

`assert is_valid_username("User123") == True`

`assert is_valid_username("12User") == False`

`assert is_valid_username("Us er") == False`

Expected Output #1:

• Username validation logic successfully passing all AI- generated test cases.

Code :

```
1 def is_valid_username(username):
2     if len(username) < 5 or len(username) > 15:
3         return False
4     if not username[0].isalnum():
5         return False
6     for char in username:
7         if not char.isalnum() and char != '_':
8             return False
9     return True
10 # Test cases for the is_valid_username function
11 assert is_valid_username("user123") == True, "Test case 1 failed"
12 assert is_valid_username("1user") == True, "Test case 2 failed"
13 assert is_valid_username("user_name") == True, "Test case 3 failed"
14 assert is_valid_username("us") == False, "Test case 4 failed"
15 print("Username validation logic successfully passing all AI-generated test cases.")
16
```

Output :

```
PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/username_validation.py
Username validation logic successfully passing all AI-generated test cases.
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

• Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it using conditional logic and loops.

• Requirements:

- o If input is an integer, classify as "Even" or "Odd".
- o If input is 0, return "Zero".
- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

`assert classify_value(8) == "Even"`

`assert classify_value(7) == "Odd"`

`assert classify_value("abc") == "Invalid Input"`

Expected Output #2:

• Function correctly classifying values and passing all test cases.

Code:

```

1 def classify_value(x):
2     if x<0:
3         return "Negative"
4     if x%2==0:
5         return "Even"
6     elif x%2!=0:
7         return "Odd"
8     if isinstance(x, str):
9         return "Invalid Input"
10    #examples for assert testcases
11    assert classify_value(8)=="Even", "Test case 1 failed"
12    assert classify_value(7)=="Odd", "Test case 2 failed"
13    assert classify_value("abc")=="Invalid Input", "Test case 3 failed"
14    print("Function correctly classifying values and passing all test cases.")

```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.

• Requirements:

- o Ignore case, spaces, and punctuation.
- o Handle edge cases such as empty strings and single characters.

Example Assert Test Cases:

`assert is_palindrome("Madam") == True`

`assert is_palindrome("A man a plan a canal Panama") == True`

`assert is_palindrome("Python") == False`

Expected Output #3:

- Function correctly identifying palindromes and passing all AI-generated tests.

Code:

```

1 def is_palindrome(text):
2     cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
3     return cleaned_text == cleaned_text[::-1]
4     # Test cases for the is_palindrome function
5     assert is_palindrome("A man, a plan, a canal panama") == True, "Test case 1 failed"
6     assert is_palindrome("Hello") == False, "Test case 2 failed"
7     assert is_palindrome("Hi") == False, "Test case 3 failed"
8     print("Function correctly identifying palindromes and passing all AI-generated tests")
9

```

Output:

```

PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiac/palindrome_text.py
Function correctly identifying palindromes and passing all AI-generated tests

```

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

- Task: Ask AI to generate at least 3 assert-based test cases for a `BankAccount` class and then implement the class.

• Methods:

- o `deposit(amount)`
- o `withdraw(amount)`
- o `get_balance()`

Example Assert Test Cases:

`acc = BankAccount(1000)`

`acc.deposit(500)`

`assert acc.get_balance() == 1500`

`acc.withdraw(300)`

`assert acc.get_balance() == 1200`

Expected Output #4:

- Fully functional class that passes all AI-generated assertions.

Code:

```

1 class BankAccount:
2     def __init__(self, account_number, balance=0):
3         self.account_number = account_number
4         self.balance = balance
5
6     def deposit(self, amount):
7         if amount > 0:
8             self.balance += amount
9             return True
10        return False
11
12    def withdraw(self, amount):
13        if 0 < amount <= self.balance:
14            self.balance -= amount
15            return True
16        return False
17    def get_balance(self):
18        return self.balance
19
20 # Test cases for the BankAccount class
21 acc = BankAccount(1000)
22 acc.deposit(500)
23 assert acc.get_balance() == 1500, "Deposit test failed"
24 acc.withdraw(1000)
25 assert acc.get_balance() == 500, "Withdraw test failed"
26 print("All test cases passed!")

```

Task Description #5 (Email ID Validation – Apply AI for Data Validation)

• Task: Use AI to generate at least 3 assert test cases for a function `validate_email(email)` and implement the function.

• Requirements:

- o Must contain `@` and `.`
- o Must not start or end with special characters.
- o Should handle invalid formats gracefully.

Example Assert Test Cases:

`assert validate_email("user@example.com") == True`

`assert validate_email("userexample.com") == False`

`assert validate_email("@gmail.com") == False`

Expected Output #5:

• Email validation function passing all AI-generated test cases and handling edge cases correctly.

Code:

```

1 def validate_email(email):
2     if not email or email[0] in "!@#%&*()-+ " or email[-1] in "!@#%&*()-+ ":
3         return False
4     if "@" in email and "." in email.split("@")[-1]:
5         return True
6     return False
7
8 # Test cases for the validate_email function
9 assert validate_email("user@example.com") == True
10 assert validate_email("userexample.com") == False
11 assert validate_email("@gmail.com") == False

```

Output :

```

PS C:\Users\PC> & C:/Users/PC/AppData/Local/Programs/Python/Python314/python.exe c:/Users/PC/Downloads/aiaac/email.py

```